Hair Cutting Robots

Lecture 5: Topic-Isomorphic Bus and the Five Worked Examples

Supervisor - Prof. Dr. Mohammed Aquil Mirza, COMP
Student Mentors - Mr. Ruan Haozhe, Mr. Jiang Suyu, Mr. Jiuyuan Zhao

June 2026
The Hong Kong Polytechnic University

Where We Are

Lecture 4 gave you Hotaru: `LServer!`, `endpoint!`, `outpoint!`, and the broker as a fifth role.

Today's promise
By the end of the hour you will read the topic table, the message schemas, and the five worked examples the same way both Desktop and Web teams read them.

Why this lecture is for everyone
The bus is not a Bus-team concern. Every team subscribes to it. If you and your twin disagree about what a topic means, you both lose.

What You Will Be Able to Do Afterwards

Read the topic table without help
Pick any row of Appendix B and explain who publishes, who subscribes, what QoS it uses, and whether it is retained.

Pick a QoS without guessing
Given a new message kind, decide between 0, 1, and 2 by what would go wrong if the message were dropped or duplicated.

Trace one of the five worked examples end-to-end
Start at the operator's command and finish at `plan/approve` or `plan/reject`, naming every topic the message hits.

1 Bus, Not Socket

The Mental Model You Already Have

Most students arrive thinking of the network as a pair of **sockets**.

A socket is a tunnel.
Two endpoints, one pipe. A writes, B reads. If you want C to hear, C needs its own pipe to A.

WebSocket is still a tunnel.
It just upgrades an HTTP connection so both sides can speak at any time. The shape is unchanged: two endpoints, one pipe.

Why the Tunnel Breaks for HCR

Imagine the initiator wants every peer to see the same twin state.

With tunnels
The initiator opens one WebSocket per peer. To add a peer, the initiator must know about it. To remove a peer, the initiator must clean up.

What this costs you
The initiator's code now knows the cardinality of the audience. Topology becomes a feature of the application instead of a feature of the network.

The Bus Is a Stadium

A bus replaces the pipe with a hall.

Publishers shout topics.
They do not name receivers. They say "this message belongs to topic `twin/state`."

Subscribers raise hands for topics.
They say "I would like everything on `twin/state`." They do not name senders.

The broker is the room.
It maps the shouters to the raised hands. Nobody on either side knows the cardinality of the other.

The Two Properties That Matter

Sender does not know receivers.
Add or remove a peer without touching the initiator. Topology is a runtime property of the room, not a compile-time property of the code.

Receiver does not know senders.
The same subscriber code works whether the publisher is the desktop initiator, the web server, the sandbox, or the audit replay tool. This is the seed of *simulator-as-gatekeeper*: the real robot is just another subscriber.

In Code: A Tunnel

A WebSocket handler in Hotaru. One peer at a time, one named connection.

```
endpoint!{
    APP.url("/ws/peer"),
    pub peer_socket <WS> {
        let mut sock = req.upgrade().await?;
        while let Some(msg) = sock.recv().await {
            sock.send(reply_for(msg)).await?;
        }
        Ok(())
    }
}
```

Add a second peer and you write a second handler — or build your own dispatch table to share this one.

In Code: A Bus

A broker plus one subscription. Any number of peers can subscribe; the publisher does not know they exist.

```
LServer!(
    BROKER = Broker::new()
        .binding("127.0.0.1:1883")
        .build()
);

endpoint!{
    BROKER.topic("twin/state/+"),
    pub on_twin <MQTT> {
        let msg: IncomingPublish = req.publish()?;
        render(msg.topic.as_ref(), msg.payload());
        Ok(())
    }
```

Tunnel vs Stadium, Side by Side

	WebSocket tunnel	MQTT bus
Addressing	by peer (URL/IP)	by topic
Fan-out	N connections	one publish
Add a peer	code change	one subscribe call
Remove a peer	code change	disconnect
Replay	re-run the sender	re-run the topic

One sentence

A tunnel is for two participants who already know each other. A bus is for a contract that does not care who shows up.

2 The Topic Table

What Is in Appendix B

Appendix B is a single table. Each row is one topic and answers four questions.

Who publishes, who subscribes, which QoS, retained or not.
These four columns are the public face of every team's slice. Your team's chapter must agree with this table or your code is wrong, not the table.

This table is frozen at the end of C2 (this lecture).
After today, adding a topic requires a one-page ARB proposal. Renaming one too.

Naming Rule: Slash-Separated Layers

A topic name is a path. Layers go from most general to most specific, left to right.

General template

```
kind/role/id/event
```

Concrete examples

```
twin/state/r01
plan/approve
plan/reject/r02
sensor/skin/r01/distance
audit/entry
```

Variable Segments: + and

Subscribers do not always want one topic; they want a *shape*.

+ matches exactly one segment
`twin/state/+` matches `twin/state/r01` and `twin/state/r02`, but not `twin/state/r01/joint`.

 # matches the rest of the path
`sensor/#` matches `sensor/skin/r01/distance`, `sensor/pose/r02`, and every other sensor topic. Must be the last segment.

Use + for fixed-shape fan-out; use # for "everything under this prefix."

Wildcards in Code

Two subscriptions, same handler shape, very different reach.

```
endpoint!{
    BROKER.topic("twin/state/+"),
    pub on_one_twin <MQTT> {
        let m = req.publish()?;
        let robot_id = m.topic.rsplit('/').next().unwrap();
        twin_cache(robot_id).apply(m.payload());
        Ok(())
    }
}

endpoint!{
    BROKER.topic("sensor/#"),
    pub on_any_sensor <MQTT> {
```

The Five Topic Families in HCR

`twin/...`: simulated state of each robot.
`plan/...`: a candidate action from the brain.
`sensor/...`: live measurements feeding the sandbox.
`audit/...`: the evidence trail behind every decision.
`sys/...`: liveness, will, presence, version.

Three letters in the prefix tell you which subsystem owns the row. If your team adds a topic that does not fit one of these five families, that is a design smell, not a feature.

A Slice of the Topic Table

Topic	Publisher	Subscriber	QoS	Retain
`twin/state/+`	initiator	all peers	0	yes
`plan/candidate`	brain	sandbox	1	no
`plan/approve`	sandbox	robot, peers	2	no
`plan/reject`	sandbox	brain, ops	2	no
`plan/confirm`	operator	sandbox	2	no
`sensor/skin/+/dist`	twin	sandbox	0	no
`audit/entry`	sandbox	audit-store	1	no
`sys/initiator/live`	initiator	all peers	1	yes
`sys/initiator/will`	broker	all peers	1	no

Read it row by row. Every column carries information; nothing is for decoration.

The Schema Behind `plan/candidate`

A topic row is one line. A schema row is the type the payload must obey.

```
#[derive(Serialize, Deserialize)]
pub struct PlanCandidate {
    pub plan_id:    Uuid,
    pub seq:        u64,
    pub robot_id:   Arc<str>,
    pub stage:      Stage,
    pub trajectory: Vec<Waypoint>,
    pub requires_confirm: bool,
}
```

Every payload on the bus is a serialised `haircut_core` struct. The IDL from Lecture 1 is what the topic table refers to.

Retained: The Sticky Message

What retained means
The broker remembers the *last* message on the topic and replays it to any subscriber who arrives *after* it was published.

When you want it
State that a late joiner must know immediately. “Which initiator is alive?” “What is the current twin state?”

When you do not
Events. “Approve this plan.” “A skin violation just happened.” Replaying an old approval to a late joiner is unsafe.

Why `twin/state` Is Retained

A peer that joins the room mid-session must catch up.

Without retention
The peer sees nothing until the next state tick fires. Until then, its viewport is blank or stale.

With retention
The broker hands the peer the most recent state immediately. The peer renders the world, then receives normal updates from there.

State is sticky. Events are not.

Why `plan/approve` Is Not Retained

An approval is a permission slip. It applies to a specific plan at a specific moment.

If we retained it
A peer joining ten minutes later receives a stale "approve." It might re-issue actuation against a plan that no longer matches reality.

Read the table this way
Retained = "what is currently true." Non-retained = "this just happened." Events live in time; state lives across it.

Retained in Code: State vs Event

The `retain` flag is one boolean on the publish call.

```
// twin/state/r01 --- sticky
broker.publish(&me, PublishPacket {
    topic:   Arc::from("twin/state/r01"),
    payload: encode(&twin_state),
    qos:     QoS::AtMostOnce,
    retain:  true,
    ..Default::default()
}).await;

// plan/approve --- ephemeral
broker.publish(&me, PublishPacket {
    topic:   Arc::from("plan/approve"),
    payload: encode(&approval),
```

3 QoS and Last Will

QoS Is a Trade Between Loss and Duplication

QoS 0 — at most once
Fire and forget. Cheap. Loses messages under load or packet drop. Never duplicates.

QoS 1 — at least once
The broker retries until it sees a `Puback`. Never loses. May duplicate.

QoS 2 — exactly once
A four-step handshake (`Publish`, `Pubrec`, `Pubrel`, `Pubcomp`). Never loses, never duplicates. Slowest.

Picking QoS for a Robotic Topic

Ask yourself two questions about every topic.

If this message is dropped, what breaks?
If the answer is "the next tick fixes it," QoS 0 is fine.

If this message is delivered twice, what breaks?
If the answer is "the receiver actuates twice," QoS 2 is mandatory. If the answer is "the receiver re-applies an idempotent update," QoS 1 is enough.

This is engineering, not religion. QoS 2 everywhere is expensive and slows the system to the rate of the worst-case handshake.

Worked QoS Picks

`twin/state/+` at QoS 0
A dropped frame is replaced by the next tick. Duplicates would just waste GPU.

`plan/candidate` at QoS 1
A dropped candidate stalls the brain. A duplicate is handled by the sandbox's plan-id check.

`plan/approve` at QoS 2
Dropping authorises nothing; the operator is annoyed. **Duplicating** actuates twice; the patient is injured. Only QoS 2 satisfies both bounds.

QoS in Code: One Field, Three Behaviors

```
fn publish_state(broker: &Broker<_>, st: TwinState) {
    broker.publish(&ME, PublishPacket {
        topic: Arc::from("twin/state/r01"),
        payload: encode(&st),
        qos:    QoS::AtMostOnce,    // 0
        retain: true,
        ..Default::default()
    });
}

fn publish_approve(broker: &Broker<_>, a: Approval) {
    broker.publish(&ME, PublishPacket {
        topic: Arc::from("plan/approve"),
        payload: encode(&a),
```

Last Will and Testament

The *will* is a message the broker promises to publish **on behalf of** a client if that client disappears.

Set it at connect time.

```
will = Some(WillMessage { topic, payload, qos, retain })
```

Fires on abrupt disconnect.
TCP drop, or the keep-alive window (1.5 times the configured interval) elapses without a ping.

Does not fire on clean disconnect.
If the client sends `Disconnect` politely, the broker assumes there was nothing left to say.

What HCR Puts in the Will

The initiator publishes a will on `sys/initiator/will` with payload `lost`.

What every peer does on receipt
Mark every twin as *unauthorised*. Refuse to render new state. Refuse to forward approvals. Display a banner.

Why this is the bus-level half of the e-stop
The physical e-stop in Chapter 7 is the operator's button. The will is the network's button. Both lead to the same fail-closed state. Lose either and a hung initiator can keep authorising cuts after it has crashed.

Will in Code: The Dead-Man's Switch

The will is attached to the `Connect` packet, not published manually.

```
let client = MqttClient::connect(ConnectOptions {
    client_id:    Arc::from("initiator-r01"),
    keep_alive:   Duration::from_secs(15),
    clean_session: true,
    will: Some(WillMessage {
        topic:   Arc::from("sys/initiator/will"),
        payload: Bytes::from_static(b"lost"),
        qos:     QoS::AtLeastOnce,
        retain:  false,
    }),
}).await?;
```

Never fires if you call `client.disconnect()` cleanly. That is the difference between a crash and a goodbye.

Keep-Alive Is Part of the Contract

What keep-alive is
A ping interval the client and broker agree on at `Connect` time.

What it costs you to pick wrong
Too short: needless reconnects, flaky tests, will fires on a slow Wi-Fi hop.
Too long: a dead initiator keeps its authority for tens of seconds while the cut goes on.

Our number
15 seconds. Will fires at 22.5. Documented in Appendix B alongside the topic table because it changes the meaning of every `sys/*/live` retained message.

4 The Five Worked Examples

What a Worked Example Is

A worked example is a triple.

Input
A scripted operator command plus a scripted sensor stream. Both come from Appendix C.

Expected output
An ordered list of MQTT messages on named topics with named payload shapes.

What you do with it
Your slice replays the input, captures the actual output, and diffs it against the expected. Pass means the contract held.

The Same Five on Both Tracks

The five examples are the executable definition of "Desktop and Web behave the same."

Happy path — everything works.
Skin violation — sandbox must reject.
Ambiguous command — sandbox must ask for confirmation.
Out-of-order arrival — sequence must be preserved.
Sensor impairment — fail-closed.

Each example produces the same expected output on Desktop and Web, modulo the names of the publishers. If they diverge, neither team "wins"; both go re-read Appendix C.

Example 1: Happy Path

Input

Operator says "trim 1cm off the back." All sensors nominal. Twin says skin is far. Pose is steady.

Expected sequence on the bus

1. `plan/candidate`: brain emits the trim plan.
2. `sensor/skin/r01/dist`: nominal, far above threshold.
3. `plan/approve`: sandbox approves.
4. `audit/entry`: paired with the constraint result that approved it.

The point of the test

If the boring case does not pass, nothing else will.

Example 2: Skin Distance Violation

Input
Same trim plan. Sensor reports skin distance below the floor halfway through the sweep.

Expected sequence

1. `plan/candidate`: brain emits the plan.
2. `sensor/skin/r01/dist`: drops below threshold.
3. `plan/reject`: sandbox emits with reason `skin_distance`.
4. `audit/entry`: includes the offending sensor sample.

What must not appear
`plan/approve`. Not now, not late, not after the sensor recovers.

Example 3: Ambiguous Command

Input

Operator says "shorter, but not too short." Brain cannot resolve the length within tolerance.

Expected sequence

1. `plan/candidate`: brain emits with a flag `requires_confirm`.
2. `plan/reject` with reason `ambiguous`, asking for clarification.
3. Operator publishes `plan/confirm` with the resolved length.
4. `plan/candidate`: brain re-emits with the resolved value.
5. `plan/approve`: sandbox approves the resolved plan.

The point

Confirm is a third state, not a retry. It is what makes "ask the operator" a first-class outcome of the sandbox.

Example 4: Out-of-Order Arrival

Input

Two plans `P1` and `P2`. The bus delivers `P2` first because the broker reordered under retransmission.

Expected sequence

1. Brain publishes `P1` then `P2`, each with a monotonic `seq`.
2. Sandbox receives `P2` first.
3. Sandbox holds `P2`, refuses to approve out of order.
4. Sandbox receives `P1`, approves `P1`, then approves `P2`.

What this catches

QoS-1 duplicate delivery, retransmit reordering, late acks. Use `seq` fields, not arrival time.

Example 5: Sensor Impairment

Input

The skin sensor stops publishing partway through the sweep. No frames on `sensor/skin/r01/dist` for longer than the staleness window.

Expected sequence

1. `plan/candidate`: brain emits.
2. `sensor/skin/r01/dist`: a few nominal frames, then silence.
3. `plan/reject` with reason `sensor_stale`.
4. `audit/entry`: records the staleness measurement that fired the reject.

Fail-closed is a bus property too.

Absence of evidence is evidence of absence *for the sandbox*.

How to Use Appendix C This Week

Replay the fixture into your slice.
Whatever your slice is. The fixture is a stream of `(timestamp, topic, payload)` tuples. Your job is to make your slice see those messages and produce the expected ones.

Diff strictly.
Topic strings, payload fields, order. "Mostly the same" is failure. The conformance harness does string-level comparison after canonicalising JSON key order.

Run the same five on your twin's track too.
Your pair-mate's code should produce the same expected sequence. If it does not, file an ARB issue against Appendix C, not against your twin.

Fixture File Shape (Appendix C)

A fixture is a JSONL stream. One line per bus event.

```
{"t":0,    "dir":"in",  "topic":"plan/candidate",
 "payload":{"plan_id":"...","seq":1,"stage":"trim",...}}
{"t":10,   "dir":"in",  "topic":"sensor/skin/r01/dist",
 "payload":{"d_mm":12.4}}
{"t":11,   "dir":"out", "topic":"plan/approve",
 "payload":{"plan_id":"...","seq":1}}
{"t":11,   "dir":"out", "topic":"audit/entry",
 "payload":{"plan_id":"...","verdict":"approve",
            "constraints":[{"name":"skin_distance","ok":true}]}}
```

dir=in is what the harness publishes. **dir=out** is what your slice must publish, in this order, with these payloads.

Replay Harness Skeleton

```
async fn run_fixture(path: &Path) -> Result<()> {
    let events = read_jsonl(path)?;
    let mut seen_out: Vec<BusEvent> = vec![];
    let probe = BROKER.attach_probe(|ev| seen_out.push(ev));

    for ev in events.iter().filter(|e| e.dir == "in") {
        BROKER.publish(&FIXTURE, ev.to_packet()).await;
        tokio::time::sleep(Duration::from_millis(ev.t)).await;
    }

    let expected: Vec<_> = events.into_iter()
        .filter(|e| e.dir == "out").collect();
    assert_eq!(canon(&seen_out), canon(&expected));
    Ok(())
`
```

5 Two Topologies, One Contract

Desktop: 1-to-N, Closed Room

Who is in the room
The initiator and a fixed set of peers who joined the session by invitation. Nobody else.

Who is in charge
The initiator. It publishes twin state, runs the sandbox, and emits approvals. Peers consume.

Why this shape
On-site, transient sessions. "Doctor house call." The room exists for ten minutes, then dissolves. Cardinality is known the moment the session starts.

Web: 1-to-many, Open Subscription

Who is in the room
A server and any number of browsers that can reach `wss://` and have credentials. Cardinality is unknown.

Who is in charge
The server. It publishes intermediate state slices and emits approvals. Browsers consume.

Why this shape
SaaS, training, remote demo. Browsers come and go. Cardinality is a runtime measurement, not a configuration.

What Changes Between Tracks

The transport layer
Desktop speaks MQTT natively over TCP, embedded in the initiator. Web speaks MQTT over WSS from the browser to a server-hosted broker.

The publisher of `twin/state/+`
Desktop: the initiator publishes raw twin state at full rate. Web: the server publishes intermediate-state deltas at a lower rate to fit browser bandwidth.

The sandbox's host
Desktop: in the initiator process. Web: on the server, never on the client. (Chapter 7 vs Chapter 8.)

Same Broker, Two Bindings

The only thing that changes between tracks is the listener.

```
// Desktop: embedded broker, local TCP
LServer!(
    BROKER = Broker::new()
        .binding("127.0.0.1:1883")
        .single_protocol(MQTT::server(MqttSafety::default()))
        .build()
);

// Web: server-side broker, MQTT over WSS
LServer!(
    BROKER = Broker::new()
        .binding("0.0.0.0:8084")
        .single_protocol(MQTT::server_over_wss(
```

What Does Not Change

The topic graph. The message schemas. The QoS table. The retained flags. The keep-alive value. The will payload. The five worked examples.

If any of those change between tracks, the project has failed.
This is what Appendix B exists to guarantee. The two tracks are different implementations of the same contract; the contract is not the union of what each track chose to do.

Where divergence creeps in
QoS semantics under reconnect. Retained delivery timing on a fresh subscribe. We will name these explicitly per topic so a number alone is never the whole story.

The Two Topologies in One Picture

	Desktop 1-to-N	Web 1-to-many
Authority	initiator	server
Membership	invited peers	open subscription
Sandbox location	initiator	server only
Transport	MQTT over TCP	MQTT over WSS
Twin publishing	full state, fast	deltas, throttled
Cardinality	known at session	runtime measurement
Topic graph	identical	
Schemas	identical	
Five fixtures	identical	

6 Recap and What to Do This Week

Recap

Bus, not Socket.
Topics, not peers. Sender does not know receivers; receiver does not know senders. That is why the real robot is just another subscriber.

Topic table is Appendix B.
Four columns: who publishes, who subscribes, QoS, retained. Frozen at the end of this lecture.

QoS is a trade between loss and duplication.
Pick by what would go wrong. `plan/approve` is QoS 2 because duplicate actuation injures the patient.

Five fixtures define "the same."
Both tracks pass the same five worked examples or the project has not met its safety claim.

What to Do Before the Next Sync

Read Appendix B once, then read it again.
If a row surprises you, do not work around it; raise it at the ARB.

Run the five fixtures against your current slice.
Even if your slice only handles two topics, you can still subscribe to the rest and verify the order of the messages that pass through you matches Appendix C.

Pair with your twin to compare QoS and retained behavior.
The drift catches we listed in Section 3.2 of the Handbook will hit you in the fingers, not in theory.

Preview of Lecture 6

Next time: the sandbox. The five constraints, the approve/reject/confirm semantics, the five-factor scheduling formula, and the audit trail.

Why we did the bus first
Because the sandbox publishes its decisions onto the bus you now understand. Without today's lecture, the next one is a stack of messages with no addresses on them.

Required reading for next week
The constraint-threshold table and the priority-formula fixture, both released within 48 hours after Lecture 6.